

EX-9 Write the Python program to implement Travelling Salesman Problem (TSP).

AIM

To develop a Python program to find the minimum cost path that visits all the cities exactly once and returns to the starting city using the Travelling Salesman Problem (TSP).

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of cities.

Step 3: Input the cost matrix representing the travelling cost between every pair of cities.

Step 4: Generate all possible routes (permutations) starting from the first city.

Step 5: Calculate the total travelling cost for each possible route, including the return to the starting city.

Step 6: Compare the cost of each route with the current minimum cost.

Step 7: Store the route if it has the minimum travelling cost.

Step 8: Display the minimum travelling cost and the optimal path.

Step 9: Stop the program.

PROGRAM CODE

```
from itertools import permutations
```

```
n = int(input("Enter number of cities: "))
```

```
print("Enter the cost matrix:")
```

```
cost = []
```

```
for i in range(n):  
    row = list(map(int, input().split()))  
    cost.append(row)
```

```
cities = list(range(1, n))
```

```
min_cost = float('inf')
```

```
best_path = ()
```

```
for path in permutations(cities):
```

```
    current_cost = 0
```

```
    k = 0
```

```
    for city in path:
```

```
        current_cost += cost[k][city]
```

```
        k = city
```

```
    current_cost += cost[k][0]
```

```
    if current_cost < min_cost:
```

```
        min_cost = current_cost
```

```
        best_path = (0,) + path + (0,)
```

```
print("\nMinimum Cost =", min_cost)
```

```
print("Best Path =", end=" ")
```

```
for city in best_path:
```

```
    print(city, end=" ")
```

OUTPUT

```
=====
Enter number of cities: 4
Enter the cost matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Minimum Cost = 80
Best Path = 0 1 3 2 0
```

RESULT

The Python program successfully implements the **Travelling Salesman Problem (TSP)** and finds the minimum travelling cost along with the optimal path that visits every city exactly once and returns to the starting city.